0074. Search a 2D Matrix

lium | Category: Binary Search | LeetCode: https://leetcode.com/problems/search

Problem

- Rows are sorted, and each row's first value is greater than the previous row's last. Is target present? Do it in O(log(mn)).

Intuition

- Reading the matrix row by row gives one sorted array of length mn.
- Index k maps to cell (k / n, k % n), so we can binary search the virtual array directly.

Brute Force

- Scan every cell: O(mn). Binary search each row: O(m log n).

Optimal Approach

- Lo = 0, hi = mn - 1.
- Mid → v = matrix[mid / n][mid % n]; compare as in standard binary search.

Worked Example

```
matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target 3.

lo=0 hi=11 mid=5 -> (1,1)=11 > 3 -> hi=4
lo=0 hi=4  mid=2 -> (0,2)=5  > 3 -> hi=1
lo=0 hi=1  mid=0 -> (0,0)=1  < 3 -> lo=1
lo=1 hi=1  mid=1 -> (0,1)=3  found
```

Java Solution

```java
public class SearchA2DMatrix {
    public boolean searchMatrix(int[][] matrix, int target) {
        int m = matrix.length, n = matrix[0].length, lo = 0, hi = m * n - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2, v = matrix[mid / n][mid % n];
            if (v == target) return true;
            if (v < target) lo = mid + 1;
            else hi = mid - 1;
        }
        return false;
    }
}
```

Complexity

- Time: O(log(mn)). Space: O(1).

Pitfalls

- Divide and mod by the number of columns n, not rows.